

# Apostila Java 3 — Controle de Fluxo, Projeto Integrado e Métodos

Terceira apostila da série. Consolida os conceitos praticados na fase de vetores e matrizes, apresenta um estudo de caso completo (Jogo da Velha) e faz a ponte para o próximo tema: Métodos. Continuação das apostilas de Fundamentos e de Random/Matrizes, seguindo a metodologia DevSuperior.

## Sumário

1. For-each (enhanced for)
2. Operador ternário
3. Validação de entrada em camadas
4. Padrões de projeto que emergem
5. Estudo de caso: Jogo da Velha
6. Prévia: Métodos (próximo tema)
7. Onde estou e o que vem a seguir

## 1. For-each (enhanced for)

Forma compacta de percorrer arrays e coleções quando você só precisa **ler** os elementos.

### Sintaxe

```
for (Tipo elemento : colecao) {  
    // usa 'elemento'  
}
```

Lê-se: "**para cada** `elemento` **dentro de** `colecao`, faça...".

### Comparação com o for tradicional

```
// Tradicional – dá acesso ao índice  
for (int i = 0; i < vetor.length; i++) {  
    System.out.println(vetor[i]);  
}  
  
// For-each – mais limpo, sem índice  
for (int valor : vetor) {  
    System.out.println(valor);  
}
```

### For-each em matriz — AINDA é aninhado

O for-each **não elimina** loops aninhados em matriz. Como matriz é "vetor de vetores", cada elemento do primeiro for-each é uma **linha inteira** (um vetor); é preciso outro for-each dentro para chegar aos números:

```
int[][] matriz = {{1, 2, 3}, {4, 5, 6}};

for (int[] linha : matriz) {    // cada 'linha' é um int[]
    for (int valor : linha) {    // aí sim, cada 'valor' é um número
        System.out.printf("%4d", valor);
    }
    System.out.println();
}
```

## Limitações (quando NÃO usar for-each)

| Tarefa                               | For-each serve? | Motivo                             |
|--------------------------------------|-----------------|------------------------------------|
| Ler / imprimir valores               | ✓ Sim           | Mais limpo                         |
| Somar todos os elementos             | ✓ Sim           | Só precisa dos valores             |
| Preencher com Scanner/Random         | ✗ Não           | Precisa de <code>vetor[i] =</code> |
| Achar maior/menor <b>com posição</b> | ✗ Não           | Precisa do índice                  |
| Isolar linha/coluna específica       | ✗ Não           | Precisa fixar um índice            |
| Modificar valores                    | ✗ Não           | For-each é somente leitura         |

⚠ **For-each é somente leitura.** Reatribuir a variável do laço ( `valor = valor * 2` ) altera só a cópia local, não o array. Para modificar, preencher ou usar índices, use o for tradicional.

**Resumo:** for-each é ferramenta de **legibilidade para leitura**, não de eliminação de aninhamento. Domine o for tradicional primeiro — ele é obrigatório para preencher, modificar e localizar posições.

## 2. Operador ternário

Operador **condicional** de três partes. Um `if/else` condensado que **produz um valor**.

### Sintaxe

```
variavel = (condicao) ? valor_se_verdadeiro : valor_se_falso;
```

- `?` separa a condição do primeiro resultado
- `:` separa os dois resultados

### Equivalência com if/else

```
// Ternário
double media = (cont > 0) ? (double) soma / cont : 0;

// Mesmo resultado com if/else
double media;
if (cont > 0) {
    media = (double) soma / cont;
} else {
    media = 0;
}
```

## Exemplos práticos

```
String status = (idade >= 18) ? "adulto" : "menor";
int maior = (a > b) ? a : b;
char jogador = (contador % 2 == 0) ? 'X' : 'O';    // alterna X/O por turno
```

💡 O último exemplo combina **módulo** ( `% 2` , par/ímpar) com **ternário** — padrão usado para alternar entre dois estados a cada iteração.

## Regra de ouro

**Ternário produz um valor. if/else executa ações.** Escolhendo qual valor atribuir → ternário. Decidindo quais comandos rodar → if/else.

## Cuidado: não aninhe

```
// ✖ Ilegível
String faixa = (n < 13) ? "criança" : (n < 18) ? "adolescente" : "adulto";

// ✔ Use if/else if para múltiplas faixas
if (n < 13) faixa = "criança";
else if (n < 18) faixa = "adolescente";
else faixa = "adulto";
```

Um ternário simples (uma condição, dois resultados) é limpo. A partir do aninhamento, volte ao `if/else if`.

## 3. Validação de entrada em camadas

Programas robustos nunca confiam na digitação do usuário. Há dois níveis.

### Nível 1 — validação de intervalo (while)

```
int escolha = sc.nextInt();
while (escolha < 1 || escolha > limite) {
    System.out.println("Digite entre 1 e " + limite);
    escolha = sc.nextInt();
}
```

Protege contra valores fora do alcance (evita `ArrayIndexOutOfBoundsException`). Mas **quebra** se o usuário digitar letra (`InputMismatchException`).

### Nível 2 — validação de tipo (try/catch + parseInt)

Lê como texto e converte, tratando qualquer entrada inválida sem travar:

```
int valor;
while (true) {
    String entrada = sc.nextLine().trim();
    try {
        valor = Integer.parseInt(entrada);
        if (valor >= 1 && valor <= limite) break;    // válido → sai
        System.out.println("Fora do intervalo.");
    } catch (NumberFormatException e) {
        System.out.println("Digite um número válido.");
    }
}
```

À prova de letra, símbolo e número fora do intervalo. Bônus: usando só `nextLine()`, evita o problema de buffer.

## Design: String vs int na entrada

| Tipo de entrada         | Ler como                                 | Motivo                                               |
|-------------------------|------------------------------------------|------------------------------------------------------|
| Menu (categorias 1/2/3) | <code>String</code>                      | Tolerante — qualquer digitação vira texto, sem crash |
| Índice num intervalo    | <code>int</code> / <code>parseInt</code> | Intenção já é numérica                               |

💡 Decisão consciente de design: onde o input é uma "categoria", `String` é mais robusto; onde é claramente numérico, `int` é mais direto. Pensar "e se o usuário digitar besteira?" é mentalidade acima de júnior.

## Curto-circuito como proteção

A **ordem** das condições importa. Verifique o intervalo **antes** de acessar a posição:

```
if (linha >= 1 && linha <= 3 && tabuleiro[linha - 1][coluna - 1] == '-') { ... }
```

Graças ao `&&` com curto-circuito, se o intervalo falhar, o Java nem tenta acessar a casa — evitando exceção. (Mesmo princípio de `s.length() != 1 && s.charAt(0)`.)

⚠ **Prefira condições positivas.** `linha >= 1 && linha <= 3` é mais legível e menos sujeito a erro que a dupla negação `!(linha < 1) && !(linha > 3)`. Bugs de limite se escondem nas negações.

## 4. Padrões de projeto que emergem

Padrões recorrentes observados nos exercícios de vetores, matrizes e no jogo:

### Separação de responsabilidades

Cada bloco de código deve fazer **uma** coisa: - **Preencher** (inicializa o estado) — uma vez - **Exibir** (lê o estado, sem alterá-lo) - **Modificar** (altera um ponto específico)

Misturar as três faz uma interferir na outra (ex.: repreencher com `'-'` a cada exibição apaga as jogadas). Separando, cada uma trabalha sem estragar as outras.

### Flags de controle

Variáveis booleanas que governam loops: `temVencedor`, `jogadaValida`, `funcionamento`. Colocadas na condição do `while` (`while (contador < 9 && !temVencedor)`), encerram o loop de forma limpa quando o estado muda — inclusive de dentro de um `switch`, onde o `break` não bastaria.

⚠ **Flags precisam de reset** quando o processo repete (ex.: validar senha em várias tentativas) — senão uma iteração "contamina" a próxima.

### Single-pass (uma passagem)

Extrair várias informações numa única varredura, em vez de percorrer os dados repetidas vezes:

```
for (...) {
    matriz[i][j] = random.nextInt(100); // preenche
    soma += matriz[i][j];               // acumula
    if (matriz[i][j] % 2 == 0) contPar++; // classifica
}
```

Base de pipelines de dados eficientes — não percorrer um dataset grande múltiplas vezes.

### Proteção contra divisão por zero

Sempre que houver divisão, pergunte "o divisor pode ser zero?". Contadores podem ser zero se a categoria não aparecer:

```
double media = (cont > 0) ? (double) soma / cont : 0;
```

Com `double`, dividir por zero não lança erro — produz `NaN / Infinity`, que polui a saída. O ternário protege em uma linha.

Contadores no lugar certo

Em loops aninhados (turnos × validação), o incremento pertence ao loop de **turnos**, não ao de validação — senão tentativas inválidas "consomem" turnos. Cada coisa no seu laço.

5. Estudo de caso: Jogo da Velha

Projeto que integra praticamente todos os fundamentos. Construído **por etapas**, testando cada uma.

Conceitos combinados

| Elemento do jogo   | Conceito aplicado                                   |
|--------------------|-----------------------------------------------------|
| Tabuleiro 3×3      | Matriz <code>char[][]</code>                        |
| Preencher / exibir | Loops aninhados                                     |
| Marcar jogada      | Acesso direto por índice <code>[i][j]</code> (O(1)) |
| Alternar X/O       | Módulo + ternário                                   |
| Repetir turnos     | While + contador                                    |
| Validar jogada     | While + curto-circuito                              |
| Encerrar ao vencer | Flag <code>temVencedor</code> na condição           |
| Detectar empate    | <code>!temVencedor</code> após o loop               |

Roteiro incremental (a ordem importa)

1. Criar e exibir o tabuleiro vazio
2. Receber e marcar uma jogada
3. Alternar X/O em loop de turnos
4. Validar jogadas (intervalo + casa livre)
5. Detectar vitória (linhas → colunas → diagonais)
6. Detectar empate

Construir por partes, testando cada uma, é a mentalidade profissional — não tentar tudo de uma vez.

Verificação de vitória (8 formas)

```
// Linhas
for (int i = 0; i < 3; i++) {
    if (t[i][0] != '-' && t[i][0] == t[i][1] && t[i][1] == t[i][2]) temVencedor = true;
}
// Colunas
for (int j = 0; j < 3; j++) {
    if (t[0][j] != '-' && t[0][j] == t[1][j] && t[1][j] == t[2][j]) temVencedor = true;
}
// Diagonal principal
if (t[0][0] != '-' && t[0][0] == t[1][1] && t[1][1] == t[2][2]) temVencedor = true;
// Diagonal secundária
if (t[0][2] != '-' && t[0][2] == t[1][1] && t[1][1] == t[2][0]) temVencedor = true;
```

💡 O `!= '-'` é crítico: sem ele, três casas vazias iguais "venceriam". Verificar "preenchido E três iguais" evita o falso positivo.

## O "cheiro" que aponta para métodos

O jogo funcional tem cerca de 90 linhas num único `main`, com: - o loop de exibir o tabuleiro **duplicado** (topo do turno e resultado final) - **quatro blocos** de verificação de vitória quase idênticos

Isso funciona, mas é frágil: mudar o formato de exibição exige alterar em dois lugares; a duplicação é onde bugs de copy-paste se escondem. Essa dor é a motivação natural para o próximo tema.

## 6. Prévia: Métodos (próximo tema)

Métodos são blocos de código nomeados e reutilizáveis. Resolvem exatamente a duplicação do jogo.

### Anatomia

```
static int somar(int a, int b) {    // 'a' e 'b' são parâmetros
    return a + b;                  // devolve um valor
}

// uso:
int resultado = somar(3, 5);       // resultado = 8
```

- **static** — por ora, todos os métodos ao lado do `main` levam isso (o porquê vem com POO)
- **tipo de retorno** — `int`, `double`, `boolean`... ou `void` (não retorna nada)
- **parâmetros** — os dados que o método recebe para trabalhar
- **return** — devolve o resultado (obrigatório se não for `void`)

### O jogo da velha refatorado (visão)

```
public static void main(String[] args) {
    char[][] tabuleiro = criarTabuleiro();
    // loop principal, chamando:
    exibirTabuleiro(tabuleiro);
    fazerJogada(tabuleiro, jogador, sc);
    if (verificarVitoria(tabuleiro)) { ... }
}

static char[][] criarTabuleiro() { /* preenche com '-' */ }
static void exibirTabuleiro(char[][] t) { /* o loop, UMA vez */ }
static boolean verificarVitoria(char[][] t) { /* as 8 checagens, UM lugar */ }
static void fazerJogada(char[][] t, char jogador, Scanner sc) { /* validação */ }
```

### O que os métodos resolvem

- **Eliminam duplicação** — a exibição fica em um lugar; muda uma vez, vale sempre.
- **Eliminam a classe de bugs de copy-paste** — sem código repetido, nada dessincroniza.
- **Deixam o `main` legível** — lê-se como um resumo do jogo.
- **Princípio DRY** (*Don't Repeat Yourself*) — não é firula acadêmica; é o que previne os erros de duplicação vistos na prática.

## 7. Onde estou e o que vem a seguir

Nível atual: júnior sólido

Sinais presentes no trabalho desta fase: - Combina múltiplas estruturas de forma coordenada (matriz + switch + validação + jogo completo) - Trata erros de entrada (try/catch) — a maioria dos iniciantes não faz - Toma decisões conscientes de design e sabe justificá-las - Pratica o ciclo "detectar bug → entender → corrigir → validar"

## O que separa do nível pleno/sênior

1. **Métodos / decomposição** — quebrar `main` longo em funções de responsabilidade única (próximo passo imediato).
2. **POO** — modelar com classes ( `Matriz` , `JogoDaVelha` ) em vez de tudo procedural.
3. **Coleções e Streams** — `List` , `Map` , `stream().average()` ; requer `Integer` vs `int` .
4. **Testes automatizados (JUnit)** — validar casos-limite de forma repetível.
5. **Padrões de projeto e arquitetura** — quando os fundamentos estiverem automáticos.

💡 Senioridade não vem de mais sintaxe — vem de **organização e arquitetura**, que é a próxima camada. O caminho é longo para todos; o que importa é a consistência ao longo do tempo, não a intensidade de um dia.

## Roteiro sugerido

**Métodos** (refatorar o jogo da velha) → **POO** (classes) → **Coleções/Streams** → **Testes** → **Padrões de projeto**.

---

*Apostila em evolução — atualizada conforme o avanço nos estudos.*

**Jader Silva** · GitHub: MONTANHA702 · LinkedIn: /in/jaderssantos/